FiTSAC: a Supersymmetric Algebra Calculator and the Custom Graph Theory Methods behind it

Nathaniel Smith

University of Maryland
nsmith30@terpmail.umd.edu

Abstract

FiTSAC (Field Theory Symbolic Algebra Calculator) is a symbolic calculator in Python that solves the 4D, N=1 Supersymmetric Algebra closure problem on a user-inputted selection of fields and corresponding Supersymmetric transformations. It achieves this through the application of a solving algorithm and a custom Graph Theory representation of the Supersymmetric tensor mathematics involved. If an Algebra does not close, the calculator will print LaTeX code that lists the terms which prevent the full closure of each field; this can include on-shell and gauge terms that the user can interpret. Optionally, the calculator can also print LaTeX code of the steps it takes, for user analysis or to be used in papers. The custom Graph Theory methods presented in this paper provide an elegant but straightforward way of identifying and turning terms into a preferred form, and are widely applicable to other theories.

1 Introduction

1.1 Motivation

Any valid Supersymmetry model must satisfy the Supersymmetry Algebra, which takes the form $\{D_a, D_b\}X = 2i\gamma^\mu{}_{ab}\partial_\mu X$ in N=1 Supersymmetry, where X is a placeholder for all the fields in the model. The Supersymmetric transformation D_a can be arbitrarily redefined by up to a constant factor; in this work we have assumed the convention for the transformation such that it gives a factor of 2i in the Supersymmetry Algebra. Verifying that this equation is true, which is also referred to as closing Supersymmetry Algebra, can be a laborious task especially for models with a high number of fields and/or complicated transformations laws. Furthermore, it can be beneficial when doing Supersymmetry research to consider many different possible models or parameters of models. These facts provide ample motivation for the development of computerized verification of the Supersymmetry Algebra closure.

This problem has existed for a long time; as such there are already programs that can handle relavant calculations, such as SCHOONSHIP [1], FORM [2], and CADABRA [3] . These

programs are very general computerized algebra systems that require a varying degree of user created additional structure and algorithms in order to automatically solve the Supersymmetry Algebra, and without the proper inputted algorithms some can fail to solve the Algebra. As such, a specialized model that deals specifically with closing the Supersymmetry Algebra is beneficial.

The code presented in this paper, FiTSAC (Field Theory Symbolic Algebra Calculator), currently is equipped to handle 4D interaction-free models with simple field symmetries for N=1 Supersymmetry, and much of the architecture of FiTSAC would transfer if/when subsequent versions of the code take it to arbitrary dimensions. Furthermore, the custom Graph Theory methods in this paper do not appear in the documentation of those other codes; these methods provide an elegant but straightforward way of turning terms into a preferred form and are also highly generalizable to other tensor-based theories, so it is valuable to report on them.

1.2 Mathematics

The mathematical structure of objects in Supersymmetry is worth noting. The gamma matrices are tensors with one spacetime index and two spinorial indices (all indices can take only four values in the 4D theory), that are traceless and satisfy the Clifford Algebra (1).

$$(\{\gamma^\mu, \gamma^\nu\})_{ab} = 2\eta^{\mu\nu} C_{ab} \tag{1}$$

This identity can be used to reorder gammas in gamma chains, which are chains of gamma group objects that are each contracted to each other. The other primary gamma group object referenced in this work is the gamma5 matrix, which has two spinorial indices.

$$(\gamma^5)_{ab} = i(\gamma^0\gamma^1\gamma^2\gamma^3)_{ab} = \frac{i}{4!}\epsilon^{\mu\nu\alpha\beta}(\gamma_\mu\gamma_\nu\gamma_\alpha\gamma_\beta)_{ab}$$

Where the Levi Civita symbol $\epsilon^{\mu\nu\alpha\beta}$ is a totally antisymmetric tensor density that equals 1 if its indices are aligned in an even permutation of 0123, equals -1 if they are in an odd permutation, and equals 0 if any two indices are the same. By a choice of convention, we take $\epsilon^{0123} = 1$, which implies that $\epsilon_{0123} = -1$ since it requires being multiplied by $\eta_{00} = -1$, $\eta_{11} = 1$, $\eta_{22} = 1$, and $\eta_{33} = 1$ in order to lower the indices, which provides an overall negative sign. Here, we have taken our Minkowski metric $\eta_{\mu\nu}$ to have the form diag(-1, 1, 1, 1).

By using the representation of gamma5 as $i\gamma^0\gamma^1\gamma^2\gamma^3$ and also making use of the Clifford Algebra, we can try to commute each of γ^0, γ^1, γ^2, γ^3 through the gamma5 and will arrive at the following.

$$\{\gamma^5, \gamma^\mu\} = 0 \tag{2}$$

There are other useful identities that can be derived from the structure of the gamma matrices and the Levi Civita symbol, many of which are used in the code. Some of these include the following. Equation (4) is from [4].

$$\epsilon^{\mu_1\mu_2\mu_3\mu_4}\epsilon_{\nu_1\nu_2\nu_3\nu_4} = -\delta^{\mu_1}_{[\nu_1}\delta^{\mu_2}_{\nu_2}\delta^{\mu_3}_{\nu_3}\delta^{\mu_4}_{\nu_4]} \tag{3}$$

$$\mathrm{tr}(\gamma^{\mu_1}...\gamma^{\mu_n}) = \sum_{k=2}^{n}(-1)^k\eta^{\mu_1\mu_k}\mathrm{tr}(\gamma^{\mu_2}...\gamma^{\mu_{k-1}}\gamma^{\mu_{k+1}}...\gamma^{\mu_n}) \tag{4}$$

$$\gamma^\mu\gamma^\nu\gamma^\alpha = \eta^{\mu\nu}\gamma^\alpha + \eta^{\nu\alpha}\gamma^\mu - \eta^{\mu\alpha}\gamma^\nu - i\epsilon^{\tau\mu\nu\alpha}\gamma_\tau\gamma^5 \tag{5}$$

$$\gamma^\mu\gamma^5 = \frac{i}{3!}\epsilon^{\mu\alpha\beta\rho}\gamma_\alpha\gamma_\beta\gamma_\rho \tag{6}$$

Fierz identities are especially useful, since they allow us to switch the spinorial indices between arbitrary tensors by projecting onto the basis that the gamma group objects form for 4×4 matrices. In our problem scope, we will only use the Fierz identities to get the free indices from the Supersymmetric transformations onto the same tensor/gamma chain. Since these free indices are effectively symmetrized by the anticommutator on the Supersymmetric transformations, our Fierz identities take a simpler form:

$$\delta^c_{(a}\delta^d_{b)} = -\frac{1}{2}(\gamma^\mu)^{cd}(\gamma_\mu)_{ab} - \frac{1}{4}(\sigma^{\mu\nu})^{cd}(\sigma_{\mu\nu})_{ab} \tag{7}$$

Where $\sigma^{\mu\nu} = \frac{i}{2}[\gamma^\mu, \gamma^\nu]$.

2 How to Use FiTSAC

The github repository for this code can be found at:

https://github.com/NathanielS-code/FiTSAC.git

There are certain limitations on the types of models can be input into the code. Namely, the model must be interaction-free, such that there is only a single field in each produced term when a Supersymmetric transformation is expanded. Also, the fields in the model must have "simple" symmetry structures such that each index on a field has some symmetry relationship with each other index of the same type on that field (type is spinorial or spacetime). For example, the symmetry structure of the Riemann tensor would not be able to be handled by the code.

In order to make use of this code, one must first create a list of Supersymmetric transformations and a list of fields that they correspond to, and then plug these in as arguments into the function `check_closure`. There are worked examples in the Appendix for reference. To make the fields, create one of the following objects for each field.

`Field(index_structure, symmetry_structure, field_type, ID)`

> The argument `index_structure` must be a nested list of the form `[[...], [...]]`, that contains the heights of spacetime indices in the first sublist, and the heights of spinorial indices in the second sublist. If a height is up, add "up" to the list, and if it is down, add "down" to the list.
>
> The argument `symmetry_structure` is a tuple which contains two lists, and is of the form `([...], [...])`. Each of those list in turn may or may not contain tuples that denote symmetries between indices. For example, to denote a symmetry between the 0 and 2 spacetime indices (indexing from 0) of a tensor, add the tuple (0, 2, "sym") to the first sublist if the indices are symmetric or (0, 2, "antisym") if the indices are antisymmetric.
>
> The argument `field_type` must be the string "B" if the field is a boson or "F" if the field is a fermion.

The argument `ID` must be an integer ranging from 1 to the total number of fields, and must correspond to the position of the field in the list of fields, indexing from that list 1.

To define a Supersymmetric transformation, you only create the expression on the expanded side of the equation, i.e. the side with the Supersymmetric transformation applied. In order to create this, you must first make the tensor objects and the relation objects on the tensors in each term (the relations being the contractions and free indices), then make a term object containing those tensors and relations as well as a coefficient, and then you can create the entire expression object which just contains the list of those terms.

When creating a tensor, there are several types that can be made. They are listed as follows.

`Eta(index_structure)`

For creating the metric. In this code, a spacetime Kronecker Delta is just the metric with one index up and the other down. `index_structure` for this tensor and the following tensors is built in the same way that it was for the Field object.

`C_matrix(index_structure)`

For creating the charge conjugation matrix. In this code, a spinorial Kronecker Delta is just a C_matrix with the first index down and the second index up.

`Gamma_mu(index_structure)`

For creating the standard gamma matrix objects.

`Gamma5(index_structure)`

For creating gamma5 objects.

`Sigma(index_structure)`

For creating the $\sigma^{\mu\nu} = \frac{i}{2}[\gamma^\mu, \gamma^\nu]$ objects.

`Levi_Civita(index_structure)`

For creating Levi Civita symbols. This can only create four-index Levi Civita symbols.

`Partial(index_structure)`

For creating partial derivative objects.

The same Field object from before is also used to create fields for usage in terms. Note that the fields used here must be separate objects from the one used in the field list. The `index_structure` argument is built in the same way for all of these tensors, it is still just a nested list of the form `[[...],[...]]`. Note that every tensor in a term must be a separate Python object, and also note that the same tensor object cannot be placed in multiple terms or expressions. If you want to create copies of a tensor, you are able to use a `.copy()` method to obtain this. The copied tensor is a distinct object from the original, and so can be put into the same term or into a different term or expression with no issue. Furthermore, when creating

a tensor object, it is necessary to keep all indices explicit. For example, you cannot create a gamma matrix that only has its spacetime index explicit and its spinorial indices hidden.

To create the "relations" in a term, which is an umbrella term for its contractions and free indices, use the following objects:

`Contraction(tensors, indices, index_type)`

> `tensors` is a list that contains the two tensors that are contracted.
>
> `indices` is a list that contains the contracted indices of the first and second `tensors` for this contraction, ordered such that the first index in indices is the contracted index of the first tensor in `tensors`, and likewise the second index in indices is the contracted index of the second tensor in `tensors`. The index labels range from from 0 to n-1 for the spinorial indices and 0 to m-1 for the spacetime indices, where n is the number of spinorial indices on the tensor and m is the number of spacetime indices on a tensor.
>
> `index_type` is either the Python string "spinorial" or "spacetime" depending on the type of contraction it is.

`Free_Index(tensor, index, index_type, ID)`

> `tensor` is the tensor that has the free index.
>
> `index` is the specific index that has the free index, with the numbering scheme of the indices the same as detailed in Contraction.
>
> `index_type` is the Python string "spinorial" or "spacetime" depending on the type of free index.
>
> `ID` is a Python integer that can take a value from 1 to the number of free indices in the term. `ID` = 2 can only be used if a free index with `ID` = 1 already exists, and `ID` = 3 can only be used if a free index with `ID` = 2 exists, and so on. During computation, the code will automatically lower all free indices, so the choice of index height on tensors with free indices does not matter particularly much.

Crucially, when creating free indices for the transformations, the free index with `ID` = 1 must be assigned to be the same index that is on the Supersymmetric transformation. The rest of the free indices can be assigned to spacetime indices or spinorial indices arbitrarily, but the `ID` numbers of each index type must increase as you move from the left indices to the right ones when looking at the field's indices on the side of the equation with the Supersymmetry transformation not expanded. There is no `.copy()` method for relations. Furthermore, each relation must be a separate object similar to how the tensors are.

Once you have created all of the tensors and relations in a term, you can then create the following object.

`Term(coefficient, tensor_list, relation_list)`

> `coefficient` is the potentially complex number that is the coefficient of the term. Use 1j to represent i, per Python's handling of imaginary numbers.

`tensor_list` is the list of the tensors in the object. Order does not matter since we are keeping all indices explicit and by our interaction-free assumption we only have a single field in the term.

`relation_list` is a list of the relations in the term. Order does not matter for this list either.

You can get the LaTeX representation of a term object by invoking the function `your_string = term_to_LaTeX(term)` and then printing `your_string`. Finally, once you have created all the terms in an expression, you can create the following object.

`Expression(term_list)`

`term_list` is a list containing all of the terms in the expression. The order of this list does not matter.

You can get the LaTeX representation of an expression object by invoking the function `your_string = expression_to_LaTeX(expression)` and then printing `your_string`. Each Supersymmetric transformation must be stored as the expression present on its expanded side, and with the rules on free indices assignment defined as above. Once you have created both the fields and the Supersymmetric transformation expressions, you can put them into lists invoke the following function.

`check_closure(field_list, transformation_list, print_level=0)`

`field_list` is a list of the different field objects, ordered by their `ID` number.

`transformation_list` is a list of the expressions on the expanded side of the Supersymmetric transformation equations, that matches the ordering of the `field_list`. In other words, the Supersymmetry transformation acting on the field at position 0 in `field_list` creates the expression that is at position 0 of `transformation_list`.

`print_level` is an optional parameter that determines how many steps the code prints to the user. The code will always print basic messages about whether each field closed and if one did not, which terms prevented closure. If `print_level` ≥ 1, the code will print the expression-level manipulations. If `print_level` ≥ 1.5, the code will print the reodered expression that it made before directly before combining like terms. If `print_level` ≥ 2, the code will also print the term-level manipulations it performed. The user can select to see only the term-level manipulations and the reodered expression from before combining terms by using `print_level` $= 0.5$. The code will not print the output of any steps that do not alter the expression, except when trying to combining like terms. Future versions of FiTSAC may add new print features, so see the repository for up-to-date options.

For the purpose of being able to be used in loops or as part of a routine in a user's script, The function returns `True` if all the fields close with no leftover terms, and `False` otherwise. This means it will return False even if those leftover terms are on-shell or gauge terms, but it will always the print the LaTeX of these extra terms so that the user can identify them.

See Appendix for examples of code being run with these objects and this function. It is possible to come up with forms for transformations that will massively blow up the runtime. For example, in a transformation rule that has many gamma5s in it, it is better to write this transformation with gamma5s instead of writing them in their expanded form as a Levi Civita and four gamma matrices. Mainly, long gamma chains combined with multiple Levi Civita symbols in a term are the types of scenarios that cause large runtimes.

3 Algorithm

The basic intuition behind how this algorithm works is that it transforms all tensors into a simplified form using identities, then it rearranges the tensors and indices in each term into a preferred form before combining like terms. If there are any remaining terms, it processes them a bit more and then attempts to combine like terms once again. The algorithm runs the same for each field in the field list, and by the first step of the algorithm, the code has already expanded the Supersymmetric transformation. The code is specifically trying to cancel all terms to make LHS - RHS = 0, where LHS is $\{D_a, D_b\}X$ but with the Supersymmetric transformations expanded, and RHS is the general form of the Supersymmetric Algebra, i.e. $2i\gamma^\mu\partial_\mu X$. The algorithm's steps are listed below. Note that when the code expanded the Supersymmetric transformations, it also collapsed all the metrics and charge conjugation matrices that it could. Furthermore, the Fierz identities (if necessary) are applied in the code before the Algorithm begins.

The Algorithm:

`expand_all_sigmas`

`reduce_gamma_squared` (for each term)

`expand_Levi_Civita_products`

The following then runs for each term:

- `reduce_eta_and_C`
- `auto_kill_term` (if it is automatically equal to 0)
- `Levi_Civita_gamma_converter`
- `reduce_gamma_squared` (for only this term)
- `reduce_eta_and_C`
- `reduce_two_gamma_two_partial`
- `reduce_eta_and_C`
- make tags
- reorder the tensors and indices per ordering of the tags

`combine_like_terms`

If there are any remaining terms in the expression, the code then runs the rest of the algorithm:

`expand_all_gamma5`

`reduce_Levi_Civita_products`

The following then runs for each term:

- `reduce_eta_and_C`
- `reduce_gamma_squared`
- `reduce_eta_and_C`
- `expand_gamma5s` (for this term only)
- `reduce_Levi_Civita_products` (for only this term)
- `reduce_eta_and_C`
- `reduce_long_chains`
- `reduce_eta_and_C`
- `auto_kill_term` (if it is automatically equal to 0)
- `reduce_gamma_squared` (for only this term)
- `reduce_eta_and_C`
- `reduce_gamma_trace`
- `reduce_eta_and_C`
- `reduce_two_gamma_two_partial`
- `reduce_eta_and_C`
- make tags
- reorder the tensors and indices per ordering of the tags

`combine_like_terms`

In practice, the code has never needed to use the second half of the algorithm, this is more of a safety net more than anything to make the code able to reduce non-cancelling long gamma chains if there were to be any.

Some notes on select steps: `reduce_two_gamma_two_partial` brings the contracted tensors $(\gamma^\mu\gamma^\nu)_{ab}\partial_\mu\partial_\nu$ to $2C_{ab}\eta^{\mu\nu}\partial_\mu\partial_\nu$. This will not happen automatically by rearranging and it does simplify the term, hence why it requires its own step.

The `Levi_Civita_gamma_converter` brings a gamma and four other gammas that are contracted to a Levi Civita symbol in the same gamma chain into a different form by equation (6) from the Mathematics section.

`reduce_gamma_squared` reduces gamma matrices with spacetime contractions if the contracted matrices are in the same gamma chain, and it also reduces pairs of gamma5s within the chain. It moves the related tensors to be adjacent to before doing so, of course. This step is taken both inside of the termwise loop and outside of it because new terms that are created during the termwise loop can spawn with gamma squared in them, and having an extra run of this step run before the termwise loop can reduce runtime.

`auto_kill_term` will trigger if, for example, there is a Levi Civita with a self contraction, or a symmetric tensor is doubly contracted to an antisymmetric tensor or two partials, or in various other cases.

The `expand_all_gamma5` step takes into account the identities that allow reduction of contracted gammas when we expand gamma5, such as multiple forms of equation (6), but also equation (8) below. It always tries to get rid of as many gamma matrices as possible when expanding gamma5.

$$\gamma^\mu\gamma^\nu\gamma^5 = \eta^{\mu\nu}\gamma^5 - \frac{i}{2}\epsilon^{\mu\nu\alpha\beta}\gamma_\alpha\gamma_\beta \tag{8}$$

`reduce_long_chains` reduces all gamma chains in a term that are length 4 or greater to less than that length by successively applying equation (5).

The make tags and reorder terms steps are where the graph theory enters. While these are only two steps, they are the most intricate out of all the steps.

4 Graph Theory Methods for Preferred Form

In order to convert each term into the same unique preferred form, we must identify what that preferred form is. We do this by creating unique "tags" for each tensor and index in a given term. These tags are Python strings that can be lexicographically ordered. Since they take a specific unique ordering, and they have a one-to-one correspondence with the tensors/indices, we can use the ordering of the tags to determine a unique final ordering for the tensors/indices.

Since we want to get the same final ordering for all combinable terms, the tags must be generated the same no matter what the initial state of the term is prior to reordering. If the tags did not generate the same in combinable terms, there could be a scenario in which two terms that can combine instead do not due to their reorderings being different.

4.1 Graph Representation

In order to generate tags that will be invariant from the initial ordering, we use a graph representation of each term, and then create the tags based on the position of each tensor within that graph. An example of such a graph can be seen in Fig 1. The free indices provide unique landmarks in the graph, and we can partially base the position of the other tensors in the graph from these free indices.

Note that when we reorder gamma matrices (and/or gamma5 matrices) using the Clifford Algebra, we are able to move spinorial indices from one gamma matrix to another. This would ordinarily break the assumption of our graph being invariant from initial ordering, as for example the terms $\gamma^\mu{}_a{}^c\gamma^\nu{}_{cb}\partial_\mu A_\nu$ and $\gamma^\nu{}_a{}^c\gamma^\mu{}_{cb}\partial_\mu A_\nu$ would then not produce the same graphs. However, this can be solved by writing each chain of gamma matrices as a single "effective tensor" in our graphs. In other words, $(\gamma^{\mu_1}\gamma^{\mu_2}...\gamma^{\mu_n})_{ab} = (G^{\mu_1\mu_2\cdots\mu_n})_{ab}$. Note that by this point in our code, charge conjugation matrices and sigma matrices have been reduced, therefore the only gamma group objects left to that can be in chains are the standard gamma matrices and gamma5. Gamma5 matrices that are in gamma chains will not contribute true spacetime indices to the effective tensor, but we will treat the effective tensor as having invisible free spacetime indices

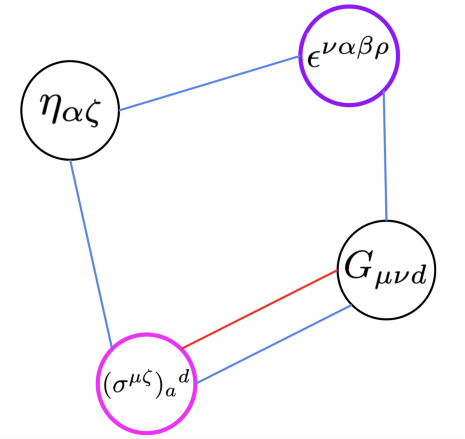


Figure 1: A graph representing the term $\eta_{\alpha\zeta}\epsilon^{\nu\alpha\beta\rho}(\sigma^{\mu\zeta})_a{}^d G_{\mu\nu d}$. The lines connecting tensors denote contractions: the red lines are spinorial contractions whereas the blue lines are spacetime contractions. The pink/purple outlines on certain tensors are due to those tensors having free indices and thus being "landmarks" with uniquely determined positions in the graph. The outlines are different hues to indicate that the free indices are different, and thus that the landmarks are different. Since we assume that all of the tensors in our theory have a symmetry relationship between each pair of indices with the same index type, the graph need not detail which index the contractions are between while we are currently only trying to reorder tensors.

for each gamma5 for later when we reorder the chain. When we reorder the chains, we may produce new terms from the Clifford Algebra, but those terms will contain less gamma matrices than the original term. When we reorder tensors in those new terms, terms that they produce will have even less gammas, and so on, such that the number of gammas in terms produced by this process will monotonically decrease. This means that we will not produce infinite terms from this reordering scheme. There is no need for additional effective tensors because no other tensors in our terms can trade indices like this and also because all tensors in our theory have some symmetry relationship between all indices. The second statement is true by the nature of the objects in 4D Supersymmetry and by our assumption of simple field symmetries. Fig 2 visualizes this in the graph representation, including the effective tensor consideration.

No matter what the order of the tensors in the term initially are, as long as the tensors and effective tensors have the same contractions and free indices, the graph will be the same. And if the tensors and effective tensors have different contractions or free indices, the terms will not be able to combine and so it is both expected and true that the graphs will not be be the same. Notice that if a tensor is contracted to multiple identical tensors each at indistinguishable positions in the graph, then arbitrarily switching which identical tensor is contracted to which index of the original tensor will not change the graph. This invariance of the graph also holds if we have a set of identical tensors at indistinguishable positions contracted to a second set of identical tensors at indistinguishable positions; swapping the positions of any of the tensors within one of these sets will not change the graph.

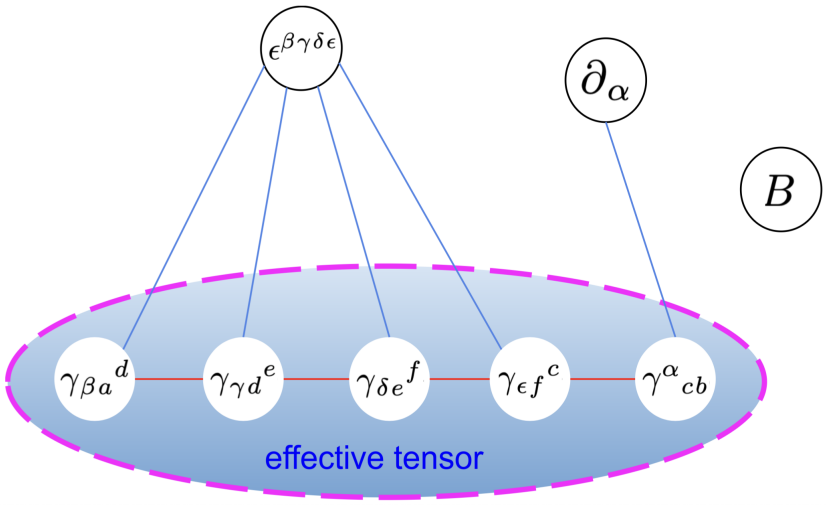


Figure 2: A graph representing the term $\frac{i}{24}\gamma^{\alpha}{}_{cb}\epsilon^{\beta\gamma\delta\epsilon}\gamma_{\beta a}{}^{d}\gamma_{\gamma d}{}^{e}\gamma_{\delta e}{}^{f}\gamma_{\epsilon f}{}^{c}\partial_\alpha B$. The five long gamma chain present in this term is being treated as one effective tensor in the graph (denoted by the dashed line area surrounding it). This is because spinorial indices can be exchanged between gamma matrices using the Clifford Algebra, so we need to combine gamma chains into an effective tensor to preserve invariance of the graph from initial ordering of the term. The effective tensor has the free indices a and b on it, so it is a landmark in the graph and thus has a pink outline.

4.2 Creating Tags for Tensors

Let us now discuss how to make tags from these graphs, specifically for the tensors first. Recall that we want the information of the position of a tensor within the graph to be stored in its tag. If the tensor has free indices, these free indices act as a landmark in the graph and so we are able to uniquely specify the position of the tensors by their free indices. We also include the tensor's type in the tag as we are going to use the tags to identify like terms later. For example, if a Minkowski metric has the free indices free3 and free5 attached to it (in any order on the term), then the tag for that tensor will be `"['free3', 'free5', 'metric']"`, where the apostrophes denote that this is a Python string. Note that the list containing 'free3', 'free5', and 'metric' has been lexicographically ordered before being turned into a string. This is done with the purpose of keeping tags the same regardless of in what order these items are added to the list. By our interaction-free assumption, we will only have one field, and so we are able to assign it the tag of "field" as that tag will uniquely determine its position within the graph. Also, it is useful to know that the code stores the type of each tensor behind the scenes as a `special_type` attribute, for example here the Minkowski metric's `special_type` was 'metric'.

If a tensor does not have any free indices and is not a field, we will chart each of the paths in the graph that originate from this tensor and put those into its tag, which will effectively store its position from the graph in the tag. See Fig 3 for a visual demonstration of the process detailed here. We trace these paths one level at a time, building up a nested list as we go which we will eventually turn into a string to become the tag.

We store the `special_type` of the start tensor (call it tensor A) and an empty sublist in

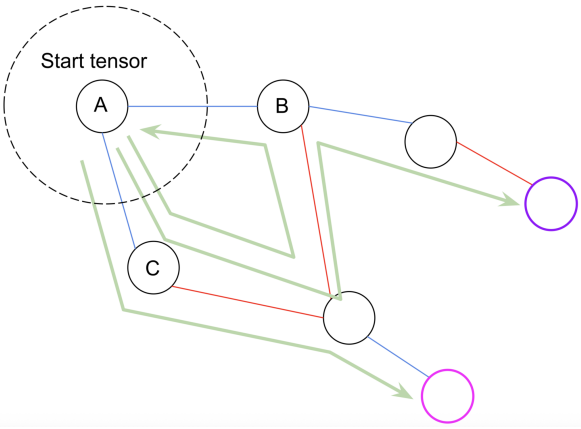


Figure 3: A diagram showing some paths coming from the start tensor A, and then stepping through the graph. Only paths that go from the start tensor immediately to through C are pictured here for visual clarity; they are tracked in light green. Each step along the graph that a path takes corresponds one level deeper in our nested list structure. The paths terminate when they reach a tensor with a free index or when they loop back to a tensor they have already visited (both cases are pictured above). It is worth noting that the code will also store seemingly trivial self-intersecting paths in the tags, for example the path A $\rightarrow$ C $\rightarrow$ A and other paths that immediately loop back on themselves are technically valid paths and will be stored in the nested list even though this is potentially redundant information. These trivial paths have been omitted from the figure for visual clarity. By tracking all paths rigorously, we are able to store all the information necessary to pinpoint the position of the start tensor in our graph.

the first level of the list, like so:

`[special_typeA, []]`

Then, inside of that sublist we store a list for the next tensors down each path, containing their `special_type`, the type of the index that they are contracted to tensor A by, the string `'c'` labeling this as a contraction, and a new sublist in which to put the subsequent level of the path. Say for example that tensor A has a spacetime contraction with tensor B and a spinorial contraction with tensor C. Then the list would become

`[special_typeA, [ [special_typeB, 'spacetime', 'c', []] , [special_typeC, 'spinorial', 'c', []] ]]`

Then, we put the same information (`special_type`, `index_type`, `'c'`, new empty sublist) for all tensors contracted to tensor B into its relevant empty sublist, and do the same for tensor C, continuing this process and recursively building up the nested structure until we hit a stop condition. Notice that each path is contained in this nested list: for example, the path that goes from tensor A to tensor B to a tensor contracted to tensor B, and so on, will be contained in subsequent layers of the nested list.

In order to prevent the nested list from becoming infinite by storing paths that loop infinitely, such as tensor A $\rightarrow$ tensor B $\rightarrow$ tensor A $\rightarrow$ tensor B $\rightarrow$ ... , we must introduce stopping conditions. There are two types of stopping conditions. If we hit a tensor with a free index on our path, we end the path. This is because the path has hit a landmark and thus

we have the sufficient information on this path for its contribution towards determining the position of our start tensor in the graph. Instead of adding `[special_type, index_type, 'c', new empty sublist]` to our nested list as we typically would for a tensor with no free indices, for a tensor with a free index we add the tag of that tensor, which will be a string. There will thus be no new empty sublist to fill in down this path, ending it.

We also stop if a path hits a tensor that has already been visited on the path. This will prevent looping paths from being stored, and if we store the information on what tensor in the path it looped back onto, means that we still have the sufficient information on this path for its contribution towards determining the position of our start tensor in the graph. The reasoning behind why this information is sufficient is that the part of the graph that a looping path would cover would have already been covered by that same path or previous branchings from that path; in addition, storing which tensor the path contracts back to specifies the exact shape of the path in the graph. If we did not store this, a path that looped back its first tensor and an identical path that looped back to its second tensor would be stored the same way in the nested list even though they are not the same. Instead of adding `[special_type, index_type, 'c', new empty sublist]` to the nested list, we add a string containing the index type as well as how far back the tensor we are looping to was in the path (the "traceback number"). Once again, we are adding a string and so there are no new sublists to continue this path down, so the path ends. The tag stops generating once every path has hit a stop condition. Every path is is guaranteed to hit at least one stop condition since the graph will have finite tensors in it, thus making it impossible to construct infinite non-self intersecting paths.

Finally, we lexicographically reorder every level of the nested list in order for the tag to be invariant from the order in which we added tensor B, tensor C, or other tensors to the list, and then we convert the whole nested list into a string, thus obtaining our tag.

4.3 Tagbumps for Tensors

Once we have made tags for every tensor in a term, it is possible for multiple of these tags to be equivalent, for example if two identical tensors are both contracted to the same tensor with a free index in the same way, or if two identical tensors have no paths that connect to tensors with free indices and their paths are indistinguishable. In order to obtain a unique lexicographic ordering of our tags, we "tagbump" equivalent tags by specific rules. This just means appending a symbol and a bump number to the end of a tensor's tag, where the symbol depends on the type of tagbump used (e.g. '#', '&', or others) and the bump number is determined by how many previous tensors with this same tag we have already tagbumped (it starts at 1). It is also worth pointing out that our terms can have multiple disjoint graphs in it, for example the term $\eta^{\alpha\beta}\eta_{\alpha\beta}\eta^{\mu\nu}\eta_{\mu\nu}$ would have a graph for the first two metrics and a graph for the second two metrics, but there would be no connections between the two graphs, thus making them disjoint.

There are two types of tagbumps that we will apply for tensors. If multiple tensors have the same tag but are in disjoint graphs, then we will tagbump all the tensors in each graph by the same bump number. This is called "outergraph" tagbumping. We tagbump everything in each graph together because if the tag of one tensor is the same as another, this implies that the tensors are in identical graphs as tags effectively store their entire graphs inside of them by storing all of their paths. In addition, if we did not bump all the tensors in a graph together,

then we could end up with situations where arbitrarily many of the tags in one graph are put at a lower bump number than the tags in a second graph, whereas the rest of the tags in the first graph are put a higher bump number than the tags in the second graph. This would break the uniqueness of the final lexicographic ordering, so we avoid that by tagbumping all the tensors of a graph together. It is technically not mathematically necessary to compute outergraph tagbumps in our problem scope due to our interaction-free assumption implying that all terms in disjoint graphs commute, but it is included for the sake of completeness and to facilitate future generalization of the code.

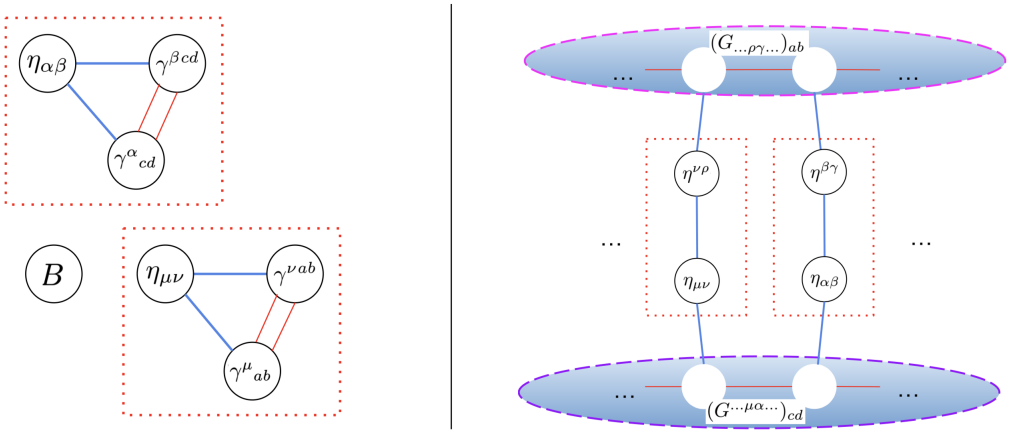


Figure 4: Outergraph tagbumps for a term are pictured on the left, and innergraph tagbumps for a different term are pictured on the right. Outergraph tagbumps occur when multiple disjoint graphs in a term are indistinguishable, and innergraph tagbumps occur when disjoint subgraphs of a graph are indistinguishable. We will have to tagbump all the tensors in the red boxes as groups.

The second type of tensor tagbumps are called "innergraph" tagbumps. These occur if multiple tensors within the same graph have the same tag. Since these tensors have the same tag, their paths will be the identical, and thus all the tags of the corresponding tensors in their identical paths will be the same. This is a trivial statement for the corresponding tensors in their paths that are the exactly same tensor, and we will not need to tagbump tensors that are shared between paths since the statement that it shares an equivalent tag with itself does not necessitate a tagbump. However, for corresponding tensors in identical paths that are not the same tensor, these will also have equivalent tags to each other and thus would need to get tagbumped too. So we tagbump all of these tensors as a group. Note that on the identical paths for those tensors, once we hit a tensor that is the exact same tensor in each path, every additional tensor past that same tensor will also show up in both paths the same. What is meant by "past" here is that the identical paths do not traverse back down each other, i.e. they take no steps any of the equivalent tensors that they originate from.

In other words, we essentially find the largest disjoint local subgraphs around each of these equivalent tensors and then outergraph tagbump those subgraphs. While it might appear that innergraph tagbumps are unnecessary since we use effective tensors and have only a single field (and so reordering our tensors/effective tensors does not change the sign of the term nor produce extra terms), it is necessary to apply this type of tagbump in order to have unique index tags as

will be covered later.

There is a scenario which may appear frightening with innegraph tagbumps: what happens if there is a fractal-like structure of local subgraphs that needs to be tagbumped at each level? Fractal-like refers to a graph that has two disjoint but identical subgraphs, and then each of those subgraphs also contains two disjoint but identical subgraphs inside, and then each of those contains two more disjoint but identical subgraphs, and so on. In this situation, it could be possible to obtain different tags depending on the order that tagbumps occur in: for example, if the deeply nested subgraphs are tagbumped before the two largest subgraphs are, then this will create different tags from the case in which the two largest subgraphs are tagbumped before the deeply nested ones are. The code accounts for this by checking for equivalent tags in successively larger distances from a given tensor (traversing down paths), so that the deepest nested subgraphs will always be tagbumped first.

This concludes our discussion on creating tags for tensors. We are now able to lexicographically order our tags and swap tensors around until the tensors match that ordering, thus bringing a term into its preferred form, up to index rearrangement.

4.4 Tags and Tagbumps for Indices

Now we will discuss tags for indices. Spacetime indices and spinorial indices are reordered separately. Indices take the tag of whatever tensor they are contracted to if they are contracted, or if they are free indices then their tag is "free1" if they are the first free index, "free2" if they are the second, and so on. When considering the gamma5s in the effective tensors, recall that we are treating them as as though they contribute invisible spacetime free indices to the effective tensor for the purposes of reordering; we assign the tag of these indices to be "gamm5 0", "gamma5 1", and so on depending on the number of gamma5 they are in the gamma chain. This does not violate the invariance of our preferred form under the initial ordering, since gamma5s commute with each other and therefore switching the positions of the gamma5s in a chain when we reorder will neither change the sign of the term nor produce any new terms. Regardless, the code has already reduced the number gamma5s in a chain down to 1 or 0 by this point in the code, so this is technically not even a necessary consideration, we only do it for the sake of completeness.

Indices may also require tagbumps if there are equivalent tags within a tensor on indices with the same index type (spinorial or spacetime). Since all free indices have a different tag (corresponding to their ID numbering), the only way that two indices will have the same tag is if they are both contracted indices. Note that we have already tagbumped the tensors by this point, so if the indices are contracted to two different tensors then they will necessarily have different tags inherited from those tensors. Therefore the only possible way for multiple indices on a tensor to have the same tag is if they are contracted to the same tensor. This includes the possibility that there could be a self contraction (from a tensor onto itself).

If these indices with equivalent tags are self contractions, then they come in pairs. We tagbump the first index of each pair with a different symbol from the second index of each pair, but we tagbump both indices with the same bump number.

If the indices with equivalent tags are not self contractions, then there are multiple

contractions between the tensor that they are on and a different tensor. In order to obtain the same final ordering of the equivalent indices on both tensors, we tagbump each contracted pair on the two tensors with the same symbol and bump number. When we reorder, these indices will then line up in matching arrangements on the two tensors.

4.5 Reordering Tensors and Indices

With index tagbumps complete, we can lexicographically reorder the index tags on each tensor (for the two index types separately) and then reorder the indices to match these types. Since we assumed simple field symmetries, and all of the other objects in our theory have some symmetry relationship between each of their indices, we will be able to perfectly match this reordering. We can also do the same with tensors.

We do have to treat the effective tensor gamma chains in a special way when reordering. First, the way we reorder tensors in the chain is by applying the Clifford algebra or the anticommutation of gamma and gamma5 to match the reordering of the corresponding spacetime indices they correspond to on the effective tensor. This is why we had gamma5 contribute invisible free indices to the effective tensor, so that we could use these to determe reordering of the gamma5s within the chain. If the initial ordering of the spin indices of a tensor effective gamma chain does not match the final reordering of the spin indices, then the only way to get the reorder the spin indices is to flip the order of the entire gamma chain and then changing the spinorial contracted index height convention between the tensors inside of the gamma chain. In other words, if this is the case, we work in reverse lexicographic order for the spacetime indices of the effective tensor gamma chain.

The code saves the tensors' tags for later use, as we can use them to determine how to match the spinorial contracted index height convention, and we can also use them to determine which terms are like terms: if the ordered list of all the tags of one term is equal to the ordered list of all the tags of another term, then we know post reordering that these terms will be like and we can combine them.

5 Conclusion

This code can be used to automatically check the Supersymmetry transformations in 4D, N=1 interaction-free theories with simple field symmetries. Examples of the code verifying the Algebra on the Chiral multiplet (multiplets from Gates et al. 2009 [5]) both off-shell and on-shell as well as the Vector multiplet from the same source can be found in the Appendix. The code will print out on shell and gauge terms when they appear for the user to identify. There are certain cases in which runtime can blow up, although the code has been written in such a way that this cases should mostly be avoided so long as the user inputs gamma5s as a single tensor instead of in the expanded form. The code can also optionally print the manipulation steps it takes at various degrees.

The Graph Theory approach present in this paper for determining a preferred form is highly generalizable to other theories or to higher dimensions. It makes only a few assumptions about our particular theory in order to function, and can be tweaked by introducing new types of effective tensors for example to accommodate other theories. Follow up work could focus on

formalizing this approach, or could focus on generalizing the code itself to higher dimensions or to different theories and/or for different types of Algebras. Much of the architecture of the code would be able to transfer over during this generalization.

6 Acknowledgements

Nathaniel Smith would like to thank Professor Sylvester James Gates and Professor Konstantinos Koutrolikos for the invaluable support and mentorship they provided during the development of this thesis, as well as the training he received from them during the 2024 SSTPRS program.

References

[1] M. Veltman and D. N. Williams. Schoonschip '91, 1993.

[2] J. Davies, T. Kaneko, C. Marinissen, T. Ueda, and J. A. M. Vermaseren. FORM Version 5.0, 2026.

[3] Kasper Peeters. Introducing Cadabra: a symbolic computer algebra system for field theory problems, 2018.

[4] V. Kaplunovsky. Traceology (course notes), 2016.

[5] S. James Gates, James Gonzales, Boanne MacGregor, James Parker, Ruben Polo-Sherk, Vincent G.J Rodgers, and Luke Wassink. 4D, N = 1 supersymmetry genomics (I). *Journal of High Energy Physics*, 2009(12):008–008, December 2009.

Appendix

A Chiral Multiplet

From [5], we have the following off-shell form of the Chiral Multiplet:

$$
\begin{aligned}
D_a A &= \psi_a \\
D_a B &= i(\gamma^5)_a{}^b \psi_b \\
D_a \psi_b &= i(\gamma^\mu)_{ab}\, \partial_\mu A - (\gamma^5\gamma^\mu)_{ab}\, \partial_\mu B - i\, C_{ab}\, F + (\gamma^5)_{ab}\, G \\
D_a F &= (\gamma^\mu)_a{}^b\, \partial_\mu \psi_b \\
D_a G &= i(\gamma^5\gamma^\mu)_a{}^b \partial_\mu \psi_b
\end{aligned}
$$

Where ψ_a is the only fermion and the rest of the fields are bosons. The on-shell formulation is derived from setting F and G to 0.

$$
\begin{aligned}
D_a A &= \psi_a \\
D_a B &= i(\gamma^5)_a{}^b \psi_b \\
D_a \psi_b &= i(\gamma^\mu)_{ab}\, \partial_\mu A - (\gamma^5\gamma^\mu)_{ab}\, \partial_\mu B
\end{aligned}
$$

The code to write for the off-shell representation is quite lengthy and just an extension of the for the on-shell representation, so it will be omitted here.

The output is (at `print_level` = 0):

```
|| Algebra satisfied for field 1

|| Algebra satisfied for field 2

|| Algebra satisfied for field 3

|| Algebra satisfied for field 4

|| Algebra satisfied for field 5

## Algebra satisfied overall
```

The output matches the result of [5], i.e. that the Supersymmetric Algebra is satisfied with no extra terms.

Now let us run the on-shell Chiral Multiplet, with a `print_level` = 1.5 this time to see some of its manipulation steps.

The setup code is as follows:

```
from fitsac import *

# setting up the fields
```

```
A = Field([[],[]], None, "B", 1)
B = Field([[],[]], None, "B", 2)
psi = Field([[],["down"]], None, "F", 3)

# setting up the right hand side of D_a A
new_psi = psi.copy()
relation = Free_Index(new_psi, 0, "spinorial", 1)
term = Term(1, [new_psi], [relation])
transformation_A = Expression([term])

# setting up the right hand side of D_a B
new_psi = psi.copy()
gamma5 = Gamma5([[],["down","up"]])
relation1 = Free_Index(gamma5, 0, "spinorial", 1)
relation2 = Contraction([gamma5, new_psi], [1, 0], "spinorial")
term = Term(1j, [gamma5, new_psi], [relation1, relation2])
transformation_B = Expression([term])

# setting up the right hand side of D_a psi_b
new_A = A.copy()
gamma = Gamma_mu([["up"],["down","down"]])
partial = Partial([["down"],[]])
relation1 = Contraction([gamma, partial], [0, 0], "spacetime")
relation2 = Free_Index(gamma, 0, "spinorial", 1)
relation3 = Free_Index(gamma, 1, "spinorial", 2)
term1 = Term(1j, [gamma, partial, new_A], [relation1, relation2, relation3])

new_B = B.copy()
gamma5 = Gamma5([[],["down", "up"]])
gamma = Gamma_mu([["up"],["down","down"]])
partial = Partial([["down"],[]])
relation1 = Contraction([gamma5, gamma], [1, 0], "spinorial")
relation2 = Contraction([gamma, partial], [0, 0], "spacetime")
relation3 = Free_Index(gamma5, 0, "spinorial", 1)
relation4 = Free_Index(gamma, 1, "spinorial", 2)
term2 = Term(-1, [gamma5, gamma, partial, new_B], [relation1, relation2, relation3, relation4])
transformation_psi = Expression([term1, term2]) # putting both terms into a single expression

# creating lists for check_closure
field_list = [A, B, psi]
transformation_list = [transformation_A, transformation_B, transformation_psi]

# verifying the closure of the Algebra
check_closure(field_list, transformation_list, print_level=1.5)
```

When this code is run, the code outputs:

```
initial:
0 = (-2)i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha B^{(1)}  + i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha
↪  B^{(1)}  + (-1) \gamma^5{}_a{}^c \gamma{}^\alpha{}_c{}_b \partial{}_\alpha B^{(2)}  + i
↪  \gamma{}^\alpha{}_b{}_a \partial{}_\alpha B^{(1)}  + (-1) \gamma^5{}_b{}^c \gamma{}^\alpha{}_c{}_a
↪  \partial{}_\alpha B^{(2)}
expression post tagmaker:
0 = (-2)i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha B^{(1)}  + i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha
↪  B^{(1)}  +  \gamma{}^\alpha{}_a{}^c \gamma^5{}_c{}_b \partial{}_\alpha B^{(2)}  + i \gamma{}^\alpha{}_a{}_b
↪  \partial{}_\alpha B^{(1)}  + (-1) \gamma^5{}_c{}_b \gamma{}^\alpha{}^c{}_a \partial{}_\alpha B^{(2)}
combined like terms:
0 =
|| Algebra satisfied for field 1
initial:
0 = (-2)i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha B^{(2)}  + (-1) \gamma^5{}_b{}^c \gamma{}^\alpha{}_a{}_c
↪  \partial{}_\alpha B^{(1)}  + (-1)i \gamma^5{}_b{}^c \gamma^5{}_a{}^d \gamma{}^\alpha{}_d{}_c
↪  \partial{}_\alpha B^{(2)}  + (-1) \gamma^5{}_a{}^c \gamma{}^\alpha{}_b{}_c \partial{}_\alpha B^{(1)}  + (-1)i
↪  \gamma^5{}_a{}^c \gamma^5{}_b{}^d \gamma{}^\alpha{}_d{}_c \partial{}_\alpha B^{(2)}
reduced gamma^2:
```

```
0 = (-2)i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha B^{(2)}  + (-1) \gamma^5{}_b{}^c \gamma{}^\alpha{}_a{}_c
↪  \partial{}_\alpha B^{(1)}  + i \partial{}_\alpha B^{(2)} \gamma{}^\alpha{}_b{}_a  + (-1) \gamma^5{}_a{}^c
↪  \gamma{}^\alpha{}_b{}_c \partial{}_\alpha B^{(1)}  + i \partial{}_\alpha B^{(2)} \gamma{}^\alpha{}_a{}_b
expression post tagmaker:
0 = (-2)i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha B^{(2)}  + (-1) \gamma^5{}_c{}_b \gamma{}^\alpha{}_a{}^c
↪  \partial{}_\alpha B^{(1)}  + i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha B^{(2)}  +  \gamma{}^\alpha{}_a{}^c
↪  \gamma^5{}_c{}_b \partial{}_\alpha B^{(1)}  + i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha B^{(2)}
combined like terms:
0 =
|| Algebra satisfied for field 2
initial:
0 = (-2)i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha F^{(3)}{}_c  + (\frac{-1}{2})i \partial{}_\alpha F^{(3)}{}_d
↪  \gamma{}^\alpha{}_e{}_c \gamma{}^\beta{}^d{}^e \gamma{}_\beta{}_a{}_b  + \frac{1}{2}i \gamma{}^\alpha{}_d{}_c
↪  \partial{}_\alpha F^{(3)}{}_e \gamma^5{}_f{}^e \gamma^5{}_g{}^d \gamma{}^\beta{}^f{}^g \gamma{}_\beta{}_a{}_b
↪  + (\frac{-1}{4})i \partial{}_\alpha F^{(3)}{}_d \gamma{}^\alpha{}_e{}_c \sigma{}^\beta{}^\gamma{}^d{}^e
↪  \sigma{}_\beta{}_\gamma{}_a{}_b  + \frac{1}{4}i \gamma{}^\alpha{}_d{}_c \partial{}_\alpha F^{(3)}{}_e
↪  \gamma^5{}_f{}^e \gamma^5{}_g{}^d \sigma{}^\beta{}^\gamma{}^f{}^g \sigma{}_\beta{}_\gamma{}_a{}_b
expanded sigmas:
0 = (-2)i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha F^{(3)}{}_c  + (\frac{-1}{2})i \partial{}_\alpha F^{(3)}{}_d
↪  \gamma{}^\alpha{}_e{}_c \gamma{}^\beta{}^d{}^e \gamma{}_\beta{}_a{}_b  + \frac{1}{2}i \gamma{}^\alpha{}_d{}_c
↪  \partial{}_\alpha F^{(3)}{}_e \gamma^5{}_f{}^e \gamma^5{}_g{}^d \gamma{}^\beta{}^f{}^g \gamma{}_\beta{}_a{}_b
↪  + \frac{1}{16}i \partial{}_\alpha F^{(3)}{}_d \gamma{}^\alpha{}_e{}_c \gamma{}^\beta{}^d{}^f
↪  \gamma{}^\gamma{}_f{}^e \gamma{}_\beta{}_a{}^g \gamma{}_\gamma{}_g{}_b  + (\frac{-1}{16})i
↪  \gamma{}^\alpha{}_d{}_c \partial{}_\alpha F^{(3)}{}_e \gamma^5{}_f{}^e \gamma^5{}_g{}^d
↪  \gamma{}^\beta{}^f{}^h \gamma{}^\gamma{}_h{}^g \gamma{}_\beta{}_a{}^i \gamma{}_\gamma{}_i{}_b  +
↪  (\frac{-1}{16})i \partial{}_\alpha F^{(3)}{}_d \gamma{}^\alpha{}_e{}_c \gamma{}^\gamma{}^d{}^f
↪  \gamma{}^\beta{}_f{}^e \gamma{}_\beta{}_a{}^g \gamma{}_\gamma{}_g{}_b  + (\frac{-1}{16})i \partial{}_\alpha
↪  F^{(3)}{}_d \gamma{}^\alpha{}_e{}_c \gamma{}^\beta{}^d{}^f \gamma{}^\gamma{}_f{}^e \gamma{}_\gamma{}_a{}^g
↪  \gamma{}_\beta{}_g{}_b  + \frac{1}{16}i \gamma{}^\alpha{}_d{}_c \partial{}_\alpha F^{(3)}{}_e
↪  \gamma^5{}_f{}^e \gamma^5{}_g{}^d \gamma{}^\gamma{}^f{}^h \gamma{}^\beta{}_h{}^g \gamma{}_\beta{}_a{}^i
↪  \gamma{}_\gamma{}_i{}_b  + \frac{1}{16}i \gamma{}^\alpha{}_d{}_c \partial{}_\alpha F^{(3)}{}_e
↪  \gamma^5{}_f{}^e \gamma^5{}_g{}^d \gamma{}^\beta{}^f{}^h \gamma{}^\gamma{}_h{}^g \gamma{}_\gamma{}_a{}^i
↪  \gamma{}_\beta{}_i{}_b  + \frac{1}{16}i \partial{}_\alpha F^{(3)}{}_d \gamma{}^\alpha{}_e{}_c
↪  \gamma{}^\gamma{}^d{}^f \gamma{}^\beta{}_f{}^e \gamma{}_\gamma{}_a{}^g \gamma{}_\beta{}_g{}_b  +
↪  (\frac{-1}{16})i \gamma{}^\alpha{}_d{}_c \partial{}_\alpha F^{(3)}{}_e \gamma^5{}_f{}^e \gamma^5{}_g{}^d
↪  \gamma{}^\gamma{}^f{}^h \gamma{}^\beta{}_h{}^g \gamma{}_\gamma{}_a{}^i \gamma{}_\beta{}_i{}_b
reduced gamma^2:
0 = (-2)i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha F^{(3)}{}_c  + (\frac{-1}{2})i \partial{}_\alpha F^{(3)}{}_d
↪  \gamma{}^\alpha{}_e{}_c \gamma{}^\beta{}^d{}^e \gamma{}_\beta{}_a{}_b  + \frac{1}{2}i \gamma{}^\alpha{}_d{}_c
↪  \partial{}_\alpha \gamma{}_\beta{}_a{}_b \gamma{}^\beta{}_e{}^d F^{(3)}{}^e  + \frac{1}{16}i
↪  \partial{}_\alpha F^{(3)}{}_d \gamma{}^\alpha{}_e{}_c \gamma{}^\beta{}^d{}^f \gamma{}^\gamma{}_f{}^e
↪  \gamma{}_\beta{}_a{}^g \gamma{}_\gamma{}_g{}_b  + \frac{1}{16}i \gamma{}^\alpha{}_d{}_c \partial{}_\alpha
↪  \gamma{}_\beta{}_a{}^e \gamma{}_\gamma{}_e{}_b \gamma{}^\gamma{}_f{}^d \gamma{}^\beta{}_g{}^f F^{(3)}{}^g  +
↪  (\frac{-1}{16})i \partial{}_\alpha F^{(3)}{}_d \gamma{}^\alpha{}_e{}_c \gamma{}^\gamma{}^d{}^f
↪  \gamma{}^\beta{}_f{}^e \gamma{}_\beta{}_a{}^g \gamma{}_\gamma{}_g{}_b  + (\frac{-1}{16})i \partial{}_\alpha
↪  F^{(3)}{}_d \gamma{}^\alpha{}_e{}_c \gamma{}^\beta{}^d{}^f \gamma{}^\gamma{}_f{}^e \gamma{}_\gamma{}_a{}^g
↪  \gamma{}_\beta{}_g{}_b  + (\frac{-1}{16})i \gamma{}^\alpha{}_d{}_c \partial{}_\alpha \gamma{}_\beta{}_a{}^e
↪  \gamma{}_\gamma{}_e{}_b \gamma{}^\beta{}_f{}^d \gamma{}^\gamma{}_g{}^f F^{(3)}{}^g  + (\frac{-1}{16})i
↪  \gamma{}^\alpha{}_d{}_c \partial{}_\alpha \gamma{}_\gamma{}_a{}^e \gamma{}_\beta{}_e{}_b
↪  \gamma{}^\gamma{}_f{}^d \gamma{}^\beta{}_g{}^f F^{(3)}{}^g  + \frac{1}{16}i \partial{}_\alpha F^{(3)}{}_d
↪  \gamma{}^\alpha{}_e{}_c \gamma{}^\gamma{}^d{}^f \gamma{}^\beta{}_f{}^e \gamma{}_\gamma{}_a{}^g
↪  \gamma{}_\beta{}_g{}_b  + \frac{1}{16}i \gamma{}^\alpha{}_d{}_c \partial{}_\alpha \gamma{}_\gamma{}_a{}^e
↪  \gamma{}_\beta{}_e{}_b \gamma{}^\beta{}_f{}^d \gamma{}^\gamma{}_g{}^f F^{(3)}{}^g
expression post tagmaker:
```

```
0 = (-2)i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha F^{(3)}{}_c  + (\frac{-1}{2})i \gamma{}^\alpha{}_e{}_c
↪  \gamma{}^\beta{}^d{}^e \gamma{}_\beta{}_a{}_b \partial{}_\alpha F^{(3)}{}_d  + (\frac{-1}{2})i
↪  \gamma{}^\alpha{}_d{}_c \gamma{}_\beta{}_a{}_b \gamma{}^\beta{}^e{}^d \partial{}_\alpha F^{(3)}{}_e  +
↪  \frac{1}{16}i \gamma{}^\alpha{}_e{}_c \gamma{}^\beta{}^d{}^f \gamma{}^\gamma{}_f{}^e \gamma{}_\beta{}_a{}^g
↪  \gamma{}_\gamma{}_g{}_b \partial{}_\alpha F^{(3)}{}_d  + (\frac{-1}{16})i \gamma{}^\alpha{}_d{}_c
↪  \gamma{}_\beta{}_a{}^e \gamma{}_\gamma{}_e{}_b \gamma{}^\gamma{}_f{}^d \gamma{}^\beta{}^g{}^f
↪  \partial{}_\alpha F^{(3)}{}_g  + \frac{1}{16}i \gamma{}^\alpha{}_e{}_c \gamma{}_\beta{}_a{}^f
↪  \gamma{}_\gamma{}_f{}_b \gamma{}^\gamma{}^e{}_g \gamma{}^\beta{}^g{}^d \partial{}_\alpha F^{(3)}{}_d  +
↪  \frac{1}{16}i \gamma{}^\alpha{}_e{}_c \gamma{}_\gamma{}_a{}^f \gamma{}_\beta{}_f{}_b \gamma{}^\beta{}^e{}_g
↪  \gamma{}^\gamma{}^g{}^d \partial{}_\alpha F^{(3)}{}_d  + (\frac{-1}{16})i \gamma{}^\alpha{}_d{}_c
↪  \gamma{}_\beta{}_a{}^e \gamma{}_\gamma{}_e{}_b \gamma{}^\beta{}^f{}^g \gamma{}^\gamma{}_g{}^d
↪  \partial{}_\alpha F^{(3)}{}_f  + (\frac{-1}{16})i \gamma{}^\alpha{}_d{}_c \gamma{}_\gamma{}_a{}^e
↪  \gamma{}_\beta{}_e{}_b \gamma{}^\gamma{}^f{}^g \gamma{}^\beta{}_g{}^d \partial{}_\alpha F^{(3)}{}_f  +
↪  \frac{1}{16}i \gamma{}^\alpha{}_e{}_c \gamma{}^\gamma{}^d{}^f \gamma{}^\beta{}_f{}^e \gamma{}_\gamma{}_a{}^g
↪  \gamma{}_\beta{}_g{}_b \partial{}_\alpha F^{(3)}{}_d  + (\frac{-1}{16})i \gamma{}^\alpha{}_d{}_c
↪  \gamma{}_\gamma{}_a{}^e \gamma{}_\beta{}_e{}_b \gamma{}^\beta{}_f{}^d \gamma{}^\gamma{}^g{}^f
↪  \partial{}_\alpha F^{(3)}{}_g  + \frac{1}{2}i \gamma{}^\alpha{}_d{}_c C{}_a{}_b \partial{}_\alpha F^{(3)}{}^d
↪  + \frac{1}{2}i \gamma{}^\alpha{}_d{}_c C{}_a{}_b \partial{}_\alpha F^{(3)}{}^d  + (\frac{-1}{2})i
↪  \gamma{}^\alpha{}_d{}_c C{}_a{}_b \partial{}_\alpha F^{(3)}{}^d  + (\frac{-1}{2})i \gamma{}^\alpha{}_d{}_c
↪  C{}_a{}_b \partial{}_\alpha F^{(3)}{}^d
combined like terms:
0 = (-2)i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha F^{(3)}{}_c  + (-1)i \gamma{}^\alpha{}_e{}_c
↪  \gamma{}^\beta{}^d{}^e \gamma{}_\beta{}_a{}_b \partial{}_\alpha F^{(3)}{}_d
|| Algebra satisfied for field 3 up to:
0 = (-2)i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha F^{(3)}{}_c  + (-1)i \gamma{}^\alpha{}_e{}_c
↪  \gamma{}^\beta{}^d{}^e \gamma{}_\beta{}_a{}_b \partial{}_\alpha F^{(3)}{}_d
## Algebra !NOT! immediately satisfied for these fields (up to gauge & on-shell terms)
```

And each output string can be copied into LaTeX to view more easily. All terms closed, except for ψ_a. By copy and pasting the extra term it did not close by into LaTeX, we get

$$(-2)i\gamma^{\alpha}{}_{ab}\partial_{\alpha}F^{(3)}{}_{c} + (-1)i\gamma^{\alpha}{}_{ec}\gamma^{\beta de}\gamma_{\beta ab}\partial_{\alpha}F^{(3)}{}_{d}$$

Which is equal to the result from [5], namely that the extra term should be:

$$-i(\gamma^{\mu})_{ab}(\gamma_{\mu}\gamma^{\nu})_{c}{}^{d}\partial_{\nu}\psi_{d}$$

These expressions are just one application Clifford Algebra and one spinorial contraction height flip apart. The code uses $F^{(3)}{}_{a} = \psi_a$, where F stands for fermion and "(3)" indicates this is the 3rd field provided by the user.

B Vector Multiplet

The off-shell Vector Multiplet from [5] is:

$$\begin{aligned} D_a A_\mu &= (\gamma_\mu)_a{}^b \lambda_b \\ D_a \lambda_b &= -i\frac{1}{4}\left([\gamma^\mu, \gamma^\nu]\right)_{ab}\left(\partial_\mu A_\nu - \partial_\nu A_\mu\right) + (\gamma^5)_{ab} d \\ D_a d &= i(\gamma^5\gamma^\mu)_a{}^b \partial_\mu \lambda_b \end{aligned}$$

And the on-shell form is:

$$D_a A_\mu = (\gamma_\mu)_a{}^b \lambda_b$$
$$D_a \lambda_b = -i\frac{1}{4}\left([\gamma^\mu, \gamma^\nu]\right)_{ab}\left(\partial_\mu A_\nu - \partial_\nu A_\mu\right)$$

Code running the off-shell case at `print_level` $= 0$ outputs:

```
|| Algebra satisfied for field 1 up to:
0 = (-2)i \gamma{}^\beta{}_a{}_b \partial{}_\alpha B^{(1)}{}_\beta
|| Algebra satisfied for field 2
|| Algebra satisfied for field 3
## Algebra !NOT! immediately satisfied for these fields (up to gauge &
↪  on-shell terms)
```

The leftover term for field 1 is

$$(-2)i\gamma^\beta{}_{ab}\partial_\alpha B^{(1)}{}_\beta$$

Which matches the leftover gauge term from [5] exactly.

Running the on-shell case at `print_level` $= 0$ produces:

```
|| Algebra satisfied for field 1 up to:
0 = (-2)i \gamma{}^\beta{}_a{}_b \partial{}_\alpha B^{(1)}{}_\beta
|| Algebra satisfied for field 2 up to:
0 = (-1)i \gamma{}^\alpha{}_a{}_b \partial{}_\alpha F^{(2)}{}_c  +
↪  (\frac{-1}{2})i \gamma{}_\beta{}_a{}_b \gamma{}^\alpha{}_d{}_c
↪  \gamma{}^\beta{}^e{}^d \partial{}_\alpha F^{(2)}{}_e  + \frac{1}{4}i
↪  \gamma{}_\gamma{}_a{}^e \gamma{}_\beta{}_e{}_b \gamma{}^\alpha{}_f{}_c
↪  \gamma{}^\gamma{}^d{}^g \gamma{}^\beta{}_g{}^f \partial{}_\alpha
↪  F^{(2)}{}_d  + i \gamma{}^\beta{}_d{}_c \gamma{}_\beta{}_a{}^e
↪  \gamma{}_\alpha{}_e{}_b \partial{}^\alpha F^{(2)}{}^d
## Algebra !NOT! immediately satisfied for these fields (up to gauge &
↪  on-shell terms)
```

The terms outputted here also match the leftover terms from [5], but some Clifford Algebra manipulations are required to make this visually obvious.